

Trinary: A Ternary (Base-3) Computer Simulation

Research Monograph

Contents

1	Introduction to Ternary Computing	5
1.1	Binary vs. Ternary Number Systems	5
1.2	Historical Context: The Setun Computer	6
1.3	Why Simulate a Ternary Computer in Software?	6
1.4	Scope of This Book	7
1.5	Verifying Your Environment	7
2	System Architecture Overview	9
2.1	Five-Layer Abstraction Stack	9
2.1.1	Module Dependency Graph	10
2.2	Memory Map	10
2.3	Number Representation	11
2.4	Dual-Stack Design	11
2.4.1	Memory Stack (PUSH / POP)	12
2.4.2	Call Stack (CALL / RET)	12
2.5	Instruction Set Summary	12
2.6	Directory Structure	12
2.7	Exploring the Public API	14
3	Environment Setup and Quick Start	15
3.1	Prerequisites	15
3.2	Package Installation	15
3.3	Running the Test Suite	16
3.4	Booting the Interactive OS Shell	16
3.5	Launching the PyQt6 Desktop UI	17
3.6	Running Demo Programs	17
3.7	Viewing ASCII Architecture Diagrams	18
3.8	Your First “Hello, Ternary” Program	18

Chapter 1

Introduction to Ternary Computing

1.1 Binary vs. Ternary Number Systems

The history of digital computing is overwhelmingly a history of binary systems. From the relay-based machines of the 1940s to modern microprocessors packing billions of transistors, the two-state “onoff” representation has dominated every era of electronic computation. This dominance is so complete that the terms “digital” and “binary” are often treated as synonymous in popular discourse. Yet binary is not the only—nor even the most efficient—radix for digital computation.

A number system’s radix r determines how many distinct symbols each digit may take. In binary ($r = 2$), digits are 0 or 1. In ternary ($r = 3$), digits may be 0, 1, or 2. The representational efficiency of a radix can be quantified by the product of its digit count d and the number of symbols per digit r , for a given representable range $R = r^d$. Minimising $r \cdot d$ for a fixed R leads to the well-known result that $r = 3$ is the most efficient integer radix [?].

Radix	Digits to represent 1000	$r \times d$
2 (binary)	10	20
3 (ternary)	7	21
4 (quaternary)	5	20
10 (decimal)	3	30

Figure 1.1: Digit efficiency comparison for $R = 1000$. Ternary requires the fewest digits, and its $r \times d$ product is competitive with binary. While binary and quaternary tie on the product metric, ternary’s per-digit information content ($\log_2 3 \approx 1.58$ bits) is closer to the theoretical optimum.

Table 1.1 illustrates this efficiency. For a range of 1000, binary needs 10 digits, decimal needs 3, but ternary requires only 7. The information-theoretic advantage was explored by Knuth [?] and Hurst [?], who observed that base-3 maximises information per symbol under the constraint of integer radices.

1.2 Historical Context: The Setun Computer

Ternary computing is not merely a theoretical curiosity. In 1958, the Soviet Union produced the Setun—the world’s only mass-produced ternary computer [?]. Designed by Nikolay Brusentsov at Moscow State University, the Setun used balanced ternary digits $(-1, 0, +1)$, implemented with ferrite-core magnetic amplifiers rather than vacuum tubes or discrete transistors.

The Setun had a 18-trit word length (representing values from $-387,420,489$ to $+387,420,489$), a 54-word ferrite-core memory, and a clock speed of roughly 200 kHz. Approximately 50 units were produced and deployed across the Soviet Union for engineering and scientific computation. Despite its technical merits—simpler arithmetic (no separate signed-number logic), lower power consumption, and elegant negation by digit-wise sign flip—the Setun was ultimately discontinued as Soviet industrial policy standardised on binary architectures.

1.3 Why Simulate a Ternary Computer in Software?

The Setun’s discontinuation was a matter of industrial politics, not technical inferiority. A ternary computer built in simulation today offers several compelling advantages:

1. **Educational value:** Building a complete ternary stack—from logic gates through ALU, CPU, assembler, and operating system—illuminates computer architecture concepts that binary-centric teaching often obscures. Students must reason about each layer from first principles rather than relying on familiar binary heuristics.
2. **Architectural exploration:** Simulation removes the fabrication constraints that limited the Setun. We can experiment with instruction sets, memory maps, display subsystems, and coprocessor designs that would be prohibitively expensive to prototype in hardware.
3. **Historical preservation:** The Setun’s design documents are sparse and mostly in Russian. A working simulator preserves and extends the intellectual legacy of ternary computing for future researchers.

4. **Novelty and engagement:** A working ternary system—complete with graphical demos, a fantasy console SDK, and a visual debugger—captures the imagination in ways that yet another binary simulator cannot.

1.4 Scope of This Book

This monograph traces the construction of a complete ternary computer system entirely in Python, from the lowest logic gates up through a functioning operating system and graphical game environment. The chapters are organised into five parts that mirror the system’s abstraction layers:

1. **Foundations** (Chapters 4–8): Trit representation, base conversion, ternary logic gates, adder circuits, and arithmetic via decimal round-trip.
2. **CPU Architecture** (Chapters 9–22): ALU design, register file, memory system, fetch-decode-execute cycle, all instruction categories (data movement, arithmetic, logical, control flow, subroutines, stack, memory access, interrupts), the two-pass assembler, and machine code encoding.
3. **Input/Output and OS** (Chapters 23–24): Memory-mapped display and keyboard, the interactive operating system shell.
4. **Advanced Subsystems** (Chapters 25–30): The PyQt6 visual simulator, games and demos, the TAL compiler, native C backend, dual display system, and Fantasy Console SDK.
5. **Hardware Simulation** (Chapters 31–34): Tensor accelerator coprocessor, cycle-accurate microarchitecture (pipeline, cache, branch predictor, DMA, bus), GPU mode, and SIMD processing.

Every chapter includes runnable code examples that readers can execute against the open-source `trinary` package. The companion codebase at <https://github.com/trinary/trinary> provides the full implementation.

1.5 Verifying Your Environment

Before proceeding, verify that your Python environment is ready. Listing 1.1 installs the package and runs a basic import test.

```
1 pip install -e .  
2 python -c "from trinary import conversion; print(  
    conversion.ternary_to_decimal('102'))"
```

Listing 1.1: Verifying the Python environment

The command should print 11, confirming that ternary-to-decimal conversion works correctly. For readers unfamiliar with base conversion, the next chapter provides a thorough treatment of the topic.

Command
<pre>python -c "from trinary.conversion import ternary_to_decimal; print(ternary_to_decimal(11))"</pre>
<pre>python -c "from trinary.conversion import ternary_to_decimal; print(ternary_to_decimal(11))"</pre>
<pre>python -c "from trinary.conversion import ternary_to_decimal; print(ternary_to_decimal(11))"</pre>
<pre>python -c "from trinary.conversion import decimal_to_ternary; print(decimal_to_ternary(11))"</pre>

Figure 1.2: Quick verification tests for the `trinary` package.

Chapter 2

System Architecture Overview

2.1 Five-Layer Abstraction Stack

The Trinary system is organised as a five-layer abstraction stack, shown in Figure 2.1. Each layer builds on the one below it and exposes a well-defined interface to the layer above.

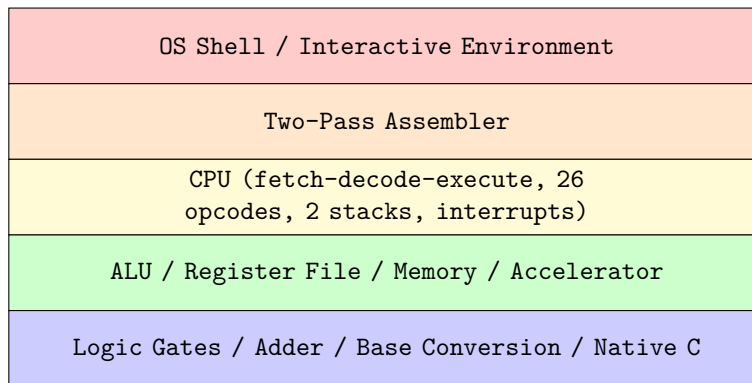


Figure 2.1: Five-layer abstraction stack of the Trinary system. Each layer encapsulates its complexity behind a clean interface.

The layers are, from lowest to highest:

1. **Gates and Foundations:** Trit representation, base conversion, ternary logic gates (TNOT, TAND, TOR), ripple-carry adder, and the optional native C backend (`libternary.so`) for accelerated operations.
2. **ALU and Memory:** The Arithmetic Logic Unit (8 operations: ADD, SUB, MUL, DIV, AND, OR, NOT, CMP), the four-register file (R0–R3), the 512-address memory, and the tensor accelerator coprocessor.

3. **CPU:** The central processing unit implementing the fetch–decode–execute cycle. Supports 26 instructions across 8 categories, dual stacks (memory-based PUSH/POP and call-list-based CALL/RET), a programmable interrupt system, timer, and optional cycle-accurate hardware simulation.
4. **Assembler:** A two-pass assembler that resolves symbolic labels and produces ternary machine-code strings.
5. **OS and Applications:** The interactive OS shell, the PyQt6 desktop graphical UI, the Fantasy Console SDK games, and user programs.

2.1.1 Module Dependency Graph

The dependency relationships among the core Python modules are shown in Figure 2.2. Arrows point from a module to its dependencies.

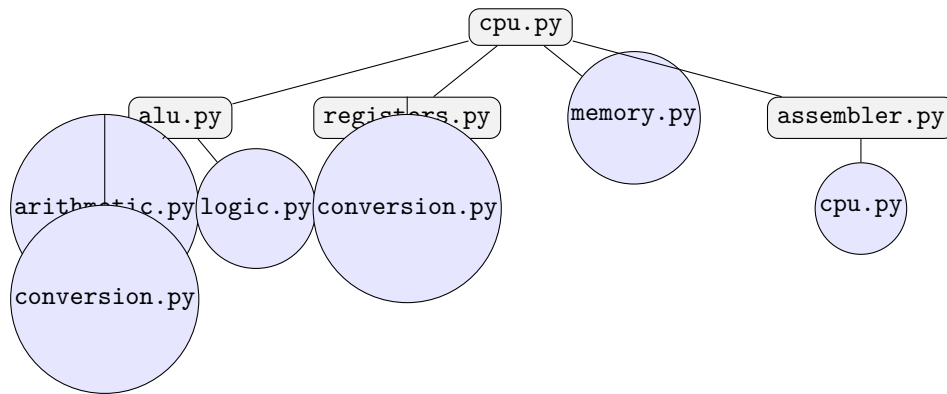


Figure 2.2: Simplified module dependency graph. `conversion.py` and `logic.py` are foundation modules with no Trinary dependencies.

2.2 Memory Map

The CPU uses a 512-address memory organised into four logical regions (Table 2.1). Addresses 0–127 hold program code and general data. Addresses 128–199 form the memory stack, growing downward from an initial stack pointer (SP) of 255. The video RAM occupies addresses 200–255, and the keyboard buffer sits at address 260.

For the Fantasy Console SDK path, an extended memory configuration with 10,000 addresses is used. This maps the SDK framebuffer (64×64 pixels) to addresses 1000–5095 and the SDK keyboard to 9000–9001.

Address range	Size	Purpose
0–127	128	General data / program storage
128–199	72	Stack (grows down from SP=255)
200–255	56	Video RAM (7 rows × 8 columns)
260	1	Keyboard buffer
261–511	251	Available

Table 2.1: Memory map for the default 512-address configuration.

2.3 Number Representation

All numeric values in the Trinary system use *signed-magnitude* ternary representation. A positive number is written as a standard base-3 numeral; a negative number is prefixed with a minus sign. For example:

- "0" = 0
- "10" = $1 \times 3^1 + 0 \times 3^0 = 3$
- "22" = $2 \times 3^1 + 2 \times 3^0 = 8$
- "-10" = -3
- "-22" = -8

This representation is *not* balanced ternary (which uses digits $-1, 0, +1$). The signed-magnitude convention was chosen for simplicity of implementation and familiarity to programmers accustomed to signed decimal notation. The conversion functions in `conversion.py` provide bidirectional mapping:

```

1 >>> from trinary.conversion import ternary_to_decimal,
   decimal_to_ternary
2 >>> ternary_to_decimal("102")
3 11
4 >>> ternary_to_decimal("-10")
5 -3
6 >>> decimal_to_ternary(11)
7 '102'
8 >>> decimal_to_ternary(-3)
9 '-10'
```

Listing 2.1: Base conversion example

2.4 Dual-Stack Design

The Trinary CPU features two independent stacks serving different purposes.

2.4.1 Memory Stack (PUSH / POP)

The memory stack occupies addresses 128–199 (72 cells). The stack pointer (SP) starts at 255, above the memory-mapped region, and grows downward. Each PUSH decrements SP and stores a register value; each POP increments SP and loads a value into a register. The stack is bounded: pushing below address 128 raises `StackOverflowError`, and popping above 255 raises `StackUnderflowError`.

```

1 # PUSH R0:  memory[SP] = R0; SP -= 1
2 # POP R0:   SP += 1; R0 = memory[SP]
3 #
4 # Initial: SP = 255
5 # After PUSH R0 (with R0=42): SP = 254, memory[255] =
   "42"
6 # After POP R1:                SP = 255, R1 = "42"

```

Listing 2.2: Memory stack operation

2.4.2 Call Stack (CALL / RET)

The CALL and RET instructions use a separate Python list (`call_stack`) rather than memory-memory stack. CALL pushes the next instruction address onto the call stack and jumps to a target; RET pops the return address and resumes execution. This separation simplifies subroutine management and avoids conflating data storage with control flow.

```

1 # CALL target: call_stack.append(PC + 1); PC = target
2 # RET:        PC = call_stack.pop()
3 #
4 # Sequence at address 10: CALL 20
5 #   call_stack = [11], PC = 20
6 # Sequence at address 20: ... RET
7 #   call_stack = [], PC = 11

```

Listing 2.3: Call stack operation

2.5 Instruction Set Summary

The Trinary CPU implements 26 instructions organised into 8 functional categories (Table 2.2). Each instruction has a ternary opcode encoding (3 trits) and a variable number of operands depending on its format. A complete reference is provided in the companion document `docs/instruction-set.md`.

2.6 Directory Structure

The `src/trinary/` package is organised as follows:

Category	Instructions	Format	Opcode range
Data Movement	LOAD, MOV, CLR	C, A, B	000–002
Arithmetic	ADD, SUB, MUL, DIV	A	010–011, 122–200
Logical	AND, OR, NOT	A, B	012–021
Comparison	CMP	A	022
Control Flow	JMP, JZ, JNZ	D	100–102
Subroutines	CALL, RET	D, E	112, 120
Stack	PUSH, POP	B	110–111
Memory Access	STOREM, LOADM	M	201–202
Interrupts	INT, IRET, EI, DI, SETIVT, SETTIMER	D, E, S, T	—
System	HALT	E	121

Table 2.2: Instruction set categories with assembly formats and ternary opcode ranges. Format codes: A (two-register), B (one-register), C (immediate), D (branch address), E (no operands), M (memory address), S (vector table), T (timer period).

```

1  src/trinary/
2      conversion.py      # Trit class + base converters
3      logic.py           # TNOT, TAND, TOR gates
4      adder.py           # Native base-3 ripple-carry adder
5      arithmetic.py      # ADD/SUB/MUL/DIV via decimal
6          round-trip
7      alu.py             # ALU: 8 operations + flag
9          computation
10     registers.py        # R0-R3 register file
11     memory.py           # RAM with write hooks
12     cpu.py              # CPU: 26 opcodes, 2 stacks,
13         interrupts
14     assembler.py        # Two-pass assembler
15     machine.py          # Machine-code encoder/decoder
16     os.py               # Interactive OS shell
17     display/            # Text + framebuffer display
18     sdk/                # Fantasy Console SDK
19     accelerator/        # Tensor coprocessor, GPU mode,
20         SIMD
21     hardware/           # Pipeline, cache, branch
22         predictor, DMA, bus
23     ui/                 # PyQt6 desktop visual simulator
24     tal.py               # TAL structured-language compiler
25     native/             # C backend (libternary.so)

```

Listing 2.4: Package directory structure

2.7 Exploring the Public API

Listing 2.5 demonstrates how to import the package and inspect its major components.

```
1 from trinary.conversion import ternary_to_decimal,
   decimal_to_ternary
2 from trinary.logic import tnot, tand, tor
3 from trinary.alu import alu
4 from trinary.registers import RegisterFile
5 from trinary.memory import Memory
6 from trinary.cpu import CPU
7
8 # Base conversion
9 assert ternary_to_decimal("22") == 8
10 assert decimal_to_ternary(8) == "22"
11
12 # Logic gates
13 assert tnot(0) == 2
14 assert tand(1, 2) == 1
15 assert tor(0, 2) == 2
16
17 # ALU
18 result = alu("ADD", "10", "12")    # 3 + 5 = 8
19 assert result == "22"
20
21 # Register file
22 regs = RegisterFile()
23 regs.load("R0", "22")
24 assert regs.store("R0") == "22"
25
26 # Memory
27 mem = Memory(512)
28 mem.store(100, "102")
29 assert mem.load(100) == "102"
30
31 # CPU
32 cpu = CPU()
33 cpu.load_program(["LOAD_R0_10", "LOAD_R1_12", "ADD_R0_R1",
34                  , "HALT"])
35 cpu.run(verbose=False)
36 assert cpu.registers.store("R0") == "22"
```

Listing 2.5: Exploring the Trinary public API

Chapter 3

Environment Setup and Quick Start

3.1 Prerequisites

The Trinary system requires Python 3.10 or later. Verify your Python version:

```
1 python --version
2 # Expected: Python 3.10.x or higher
```

Listing 3.1: Checking Python version

Additional dependencies include `setuptools` for installation (bundled with Python), and optionally `PyQt6` for the graphical desktop interface and `pygame` for the Fantasy Console SDK audio/video.

3.2 Package Installation

Install the `trinary` package in editable mode from the project root. This links the source directory directly into your Python environment, so any edits to the source are immediately reflected without reinstallation.

```
1 cd trinary
2 pip install -e .
```

Listing 3.2: Installing the trinary package

On Unix-like systems, the native C backend can be built for accelerated ALU operations:

```
1 make -C src/trinary/native
2 # or: bash build_native.sh
```

Listing 3.3: Building the native C backend (optional)

3.3 Running the Test Suite

The project includes an extensive test suite covering every subsystem. Run all tests with pytest:

```
1 python -m pytest tests/ -v
```

Listing 3.4: Running the full test suite

This executes hundreds of tests across the conversion, logic, ALU, CPU, assembler, display, OS, SDK, accelerator, hardware simulation, and native backend modules. A successful run produces output similar to Figure 3.1.

```
1 tests/test_conversion.py ..... [ 2%]
2 tests/test_logic.py ..... [ 4%]
3 tests/test_alu.py ..... [ 6%]
4 tests/test_arithmetic.py ..... [ 9%]
5 tests/test_cpu.py ..... [ 16%]
6 ...
7 tests/test_sdk.py ..... [ 85%]
8 tests/test_accelerator.py ..... [ 95%]
9 tests/test_realistic_cpu.py ..... [100%]
10 === 572 passed in 12.34s ===
```

Listing 3.5: Sample test output (abbreviated)

Figure 3.1: Abbreviated output from the full test suite.

Subsets of tests can be run by module or keyword:

```
1 python -m pytest tests/test_cpu.py -v # CPU-
   only tests
2 python -m pytest tests/test_sdk.py -v # SDK
   tests
3 python -m pytest tests/ -k "accelerator" -v # All
   accelerator tests
```

Listing 3.6: Running specific test subsets

3.4 Booting the Interactive OS Shell

The legacy operating system shell provides an interactive environment running directly on the ternary CPU. Launch it with:

```
1 python -m trinary.os
```

Listing 3.7: Booting the interactive OS shell

Upon boot, the OS clears the display, prints a banner showing the system version, and presents a command prompt (Figure 3.2). The shell accepts commands typed at the keyboard buffer (address 260) and supports HELP, CLS, REGS (show registers), MEMDUMP, and RUN (execute a stored program).


```

1 Trinary OS v1.3.0
2 Ternary (Base-3) Computer System
3 512-word memory, 26 instructions, 2 stacks
4 Type HELP for commands
5 > HELP
6 Commands: HELP, CLS, REGS, MEMDUMP, RUN, HALT
7 >

```

Figure 3.2: OS shell boot screen and HELP output.

3.5 Launching the PyQt6 Desktop UI

The graphical visual simulator provides a complete debugging environment with syntax-highlighted assembly editor, register/memory inspectors, pipeline and cache visualizers, and performance dashboards. Launch it with:

```

1 pip install pyqt6          # if not already installed
2 python -m trinary.ui.app

```

Listing 3.8: Launching the PyQt6 desktop UI

The main window (Figure ??) is organised into resizable panels. The left pane contains the assembly editor; the right pane shows machine code, registers, and flags. Lower panels display memory contents, stack state, execution trace, and the pixel framebuffer. A toolbar provides Assemble, Run, Step Into, Step Over, Reset, Pause, and Continue controls. A dropdown selector offers 13 built-in demo programs.

3.6 Running Demo Programs

The `demo_programs` module provides pre-written assembly programs demonstrating various CPU features:

```

1 python -m trinary.demo_programs

```

Listing 3.9: Running demo programs

The launcher presents a numbered menu. Available demos include:

- **Countdown:** Simple loop with decrement and conditional jump
- **Factorial:** Recursive factorial using CALL/RET
- **Fibonacci:** Iterative Fibonacci sequence
- **Timer Demo:** Program demonstrating the interrupt timer
- **Graphics Demo:** Drawing to the memory-mapped display

For the Fantasy Console SDK games (which use the 64×64 pixel framebuffer), use the `demo_games` launcher:

```

1 python -m trinary.demo_games pong          # Classic Pong
2 python -m trinary.demo_games snake          # Snake (TAL-
    compiled CPU assembly)
3 python -m trinary.demo_games breakout       # Breakout clone
4 python -m trinary.demo_games particles      # Particle
    system demo
5 python -m trinary.demo_games bouncing_logo  # Bouncing
    DVD-style logo
6 python -m trinary.demo_games tilemap        # Tilemap
    scroller
7 python -m trinary.demo_games rpg            # RPG overworld
    movement

```

Listing 3.10: Running Fantasy Console games

3.7 Viewing ASCII Architecture Diagrams

The `diagrams` module prints system architecture diagrams directly to the terminal:

```

1 python -m trinary.diagrams

```

Listing 3.11: Viewing architecture diagrams

This displays a visual overview of the CPU architecture, memory layout, fetch-decode-execute cycle, and the complete system stack—providing a quick reference for the concepts covered in Chapter 2.

3.8 Your First “Hello, Ternary” Program

Listing 3.12 demonstrates the complete workflow: writing assembly source, assembling it into machine code, loading it onto the CPU, and executing it.

```

1 from trinary.cpu import CPU
2 from trinary.assembler import Assembler
3
4 # Write assembly source
5 source = """
6 start:
7     LOAD R0 10          # R0 = 3 (ternary "10")
8     LOAD R1 12          # R1 = 5 (ternary "12")
9     ADD R0 R1           # R0 = 3 + 5 = 8 (ternary "22")
10    STOREM 100 R0       # Store result at address 100
11    HALT
12    """
13
14 # Assemble

```

```

15 assembler = Assembler()
16 program = assembler.assemble(source)
17 print("Assembled program:", program)
18
19 # Load and run
20 cpu = CPU()
21 cpu.load_program(program)
22 cpu.run(verbose=True)
23
24 # Inspect result
25 result = cpu.memory.load(100)
26 print(f"Result at memory[100]: {result}")
27 # Expected: "22" (ternary for 8)

```

Listing 3.12: Writing, assembling, and running a “Hello, Ternary” program

When executed, the verbose mode prints a step-by-step trace:

Step	PC	Instruction	R0	R1	R2	R3
0	0	LOAD R0 10	3	0	0	0
1	1	LOAD R1 12	3	5	0	0
2	2	ADD R0 R1	8	5	0	0
3	3	STOREM 100 R0	8	5	0	0
4	4	HALT	8	5	0	0
Result at memory[100]: 22						

Listing 3.13: Execution trace for the Hello Ternary program

This simple program demonstrates four instruction categories in action: data movement (LOAD), arithmetic (ADD), memory access (STOREM), and system control (HALT). The following chapters will examine each of these categories, and every CPU mechanism, in detail.